

Отчет по лабораторной работе №6

Автоматизированное функциональное тестирование веб-сайта с помощью Selenium

1. Описание предмета тестирования

Объект тестирования: веб-сайт <https://2ip.io/>.

Назначение сайта: сервис для получения информации об IP-адресе пользователя, проверки скорости интернет-соединения, проверки доменных имен, уровня анонимности и других сетевых инструментов.

1.1 Основные функции для пользователя:

- Отображение текущего IP-адреса и данных о провайдере/геолокации,
- Тестирование скорости входящего и исходящего трафика,
- Проверка доступности портов и DNS-записей,
- Проверка наличия IP-адреса в спам-базах.

2. Описание окружения тестирования

В ходе выполнения работы использовалось следующее аппаратное и программное обеспечение:

Компонент	Описание
Операционная система	Windows 11
Браузер	Google Chrome (Версия 120+)
Инструмент автоматизации	Selenium WebDriver (Python)
Библиотеки	selenium, webdriver-manager
Антивирусное ПО	Windows Defender
Дополнительно	ChromeOptions

3. Тест-кейсы

Ниже приведены описания сценариев тестирования и программная реализация на языке Python.

3.1. Тест-кейс №1: Проверка загрузки главной страницы (Позитивный)

Цель: убедиться, что главная страница доступна и содержит базовую информацию.

Ожидаемый результат: на странице присутствует текст "IP".

```
def test_main_page():
    driver = get_driver()
    try:
        driver.get("https://2ip.io/")
        WebDriverWait(driver, 10).until(
            EC.text_to_be_present_in_element((By.TAG_NAME,
"body"), "IP")
        )
        take_screenshot(driver, "test1_main_page")
    finally:
        driver.quit()
```

3.2. Тест-кейс №2: Переход в раздел проверки скорости (Позитивный)

Цель: Проверка навигации по сайту через URL.

Ожидаемый результат: URL содержит подстроку speed.

```
def test_speed_page():
    driver = get_driver()
    try:
        driver.get("https://2ip.io/speed/")
        WebDriverWait(driver, 8).until(lambda d: "speed" in
d.current_url)
        take_screenshot(driver, "test2_speed_page")
    finally:
        driver.quit()
```

3.3. Тест-кейс №3: Обработка несуществующей страницы (Негативный)

Цель: Проверка реакции системы на ввод некорректного URL.

Ожидаемый результат: Текст на странице содержит "404" или происходит редирект.

```
def test_404_page():
    driver = get_driver()
    try:
        driver.get("https://2ip.io/invalidpage")
        WebDriverWait(driver, 10).until(
            lambda d: "404" in d.page_source or d.current_url !=
"https://2ip.io/invalidpage"
        )
        take_screenshot(driver, "test3_404")
    finally:
        driver.quit()
```

3.4. Тест-кейс №4: Проверка DNS-записей домена (Комплексный)

Цель: проверка функциональности формы ввода и получения результата.

Ожидаемый результат: после ввода "google.com" и нажатия кнопки отображаются записи типа A, NS или MX.

```

def test_dns_page_error():
    """Тест страницы DNS checker"""
    driver = get_driver(headless=False) # Оставляем видимым для
отладки
    try:
        url = f"{BASE_URL}dns-checker/"
        print(f"Открываем: {url}")
        driver.get(url)

        time.sleep(5) # Даём время на загрузку (сайт тяжёлый)

        take_screenshot(driver, "test4_dns_before")

        print(f"Заголовок: {driver.title}")
        print(f"URL: {driver.current_url}")

        # === Ищем элементы отправки формы (input type=submit) ===
        submit_elements = driver.find_elements(By.XPATH,
        "//*[@type='submit']")
        print(f"Найдено                                input[type='submit']:
        {len(submit_elements)}")

        for i, el in enumerate(submit_elements):
            value = el.get_attribute("value") or
el.get_attribute("name") or "No value"
            print(f"Submit {i + 1}: value='{value}' |
visible={el.is_displayed()}")

        # Берём первый видимый submit
        button = None
        for el in submit_elements:
            if el.is_displayed() and el.is_enabled():
                button = el
                break

        if not button:
            # Альтернативный поиск любой кнопки/ссылки
            button = driver.find_element(By.XPATH,
            "//*[contains(text(), 'Проверить') or contains(text(), 'Check')]")

        # Заполняем домен
        try:
            domain_input = driver.find_element(By.XPATH,

            "//*[@type='text' and
            not(@type='hidden')]")
            domain_input.clear()
            domain_input.send_keys("google.com")
            print("Ввели домен: google.com")
        except:
            print("Не удалось найти поле для домена")

```

```

        # Кликаем
        button.click()
        print("Клик выполнен")

        # Ждём результат
        WebDriverWait(driver, 20).until(
            lambda d: any(word in d.page_source.lower() for word
in
                                ["error", "ошибк", "result",
"результат", "ttl", "record", "a ", "aaaa", "ns ", "mx "])
        )

        take_screenshot(driver, "test4_dns_success")
        print("Тест DNS страницы пройден!")

    except Exception as e:
        take_screenshot(driver, "test4_dns_error_FAILED")
        print(f"Ошибка: {type(e).__name__}: {e}")
        raise
    finally:
        driver.quit()

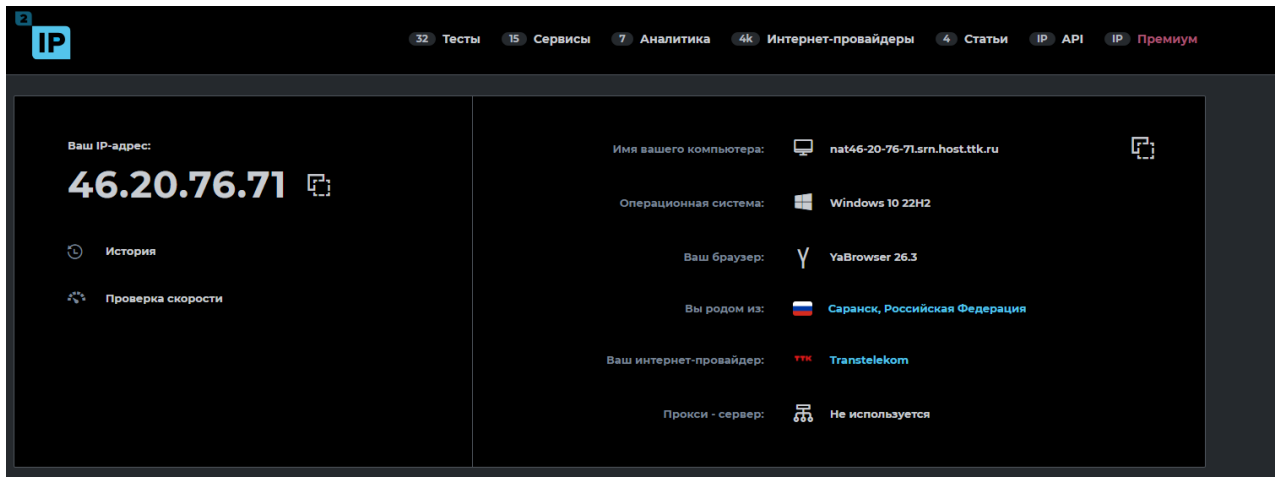
```

4. Логи и результаты автоматического тестирования

Выполнение тестов проводилось в автоматическом режиме. Ниже представлены результаты и описание действий.

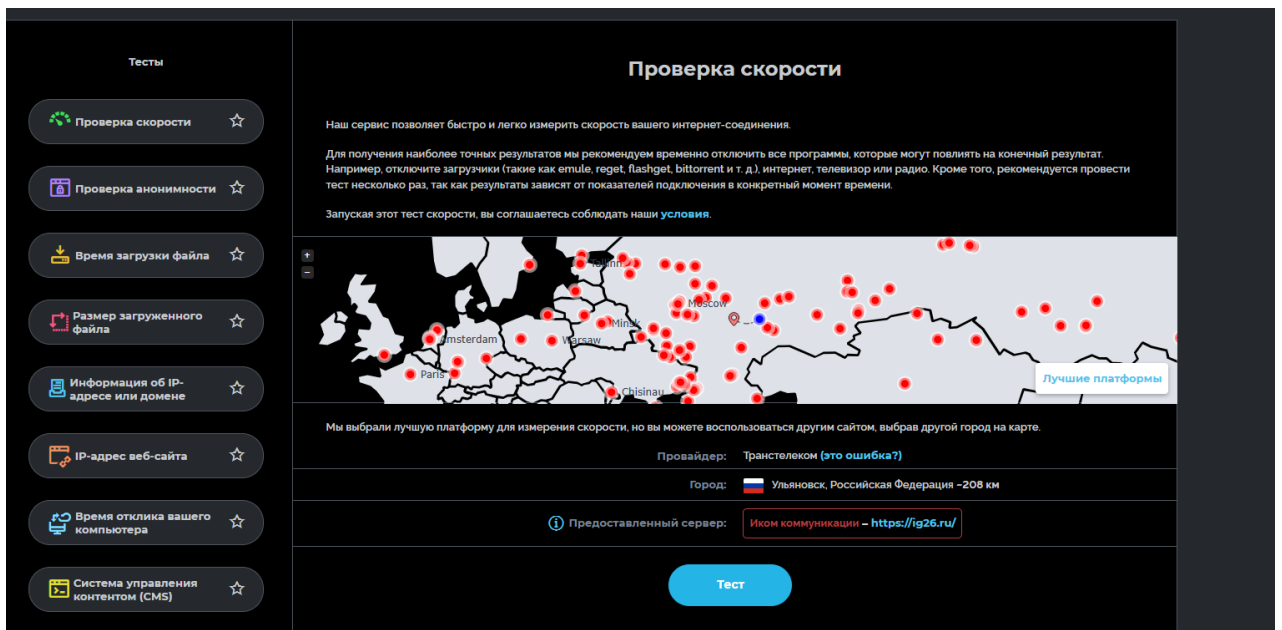
Ход тестирования:

Тест 1: Браузер успешно открыл 2ip.io. Элементы загрузились в течение 3-х секунд. Статус: PASSED.



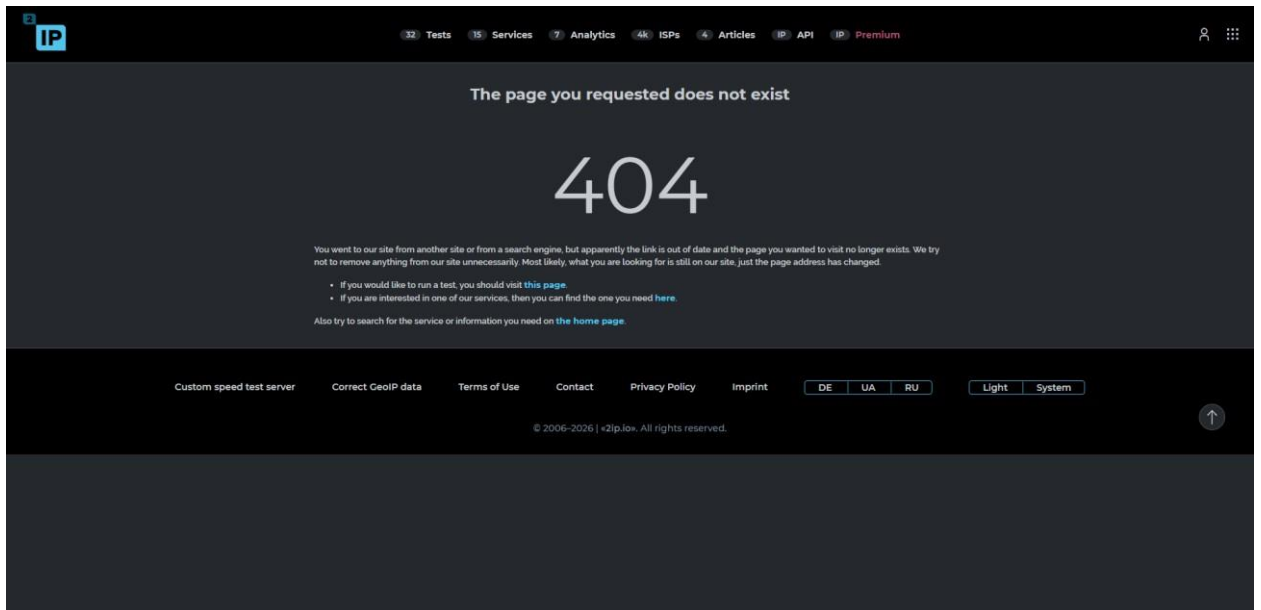
Скриншот: test1_main_page.png

Тест 2: Выполнен переход на страницу /speed/. Заголовок страницы подтвердил корректность перехода. Статус: PASSED.



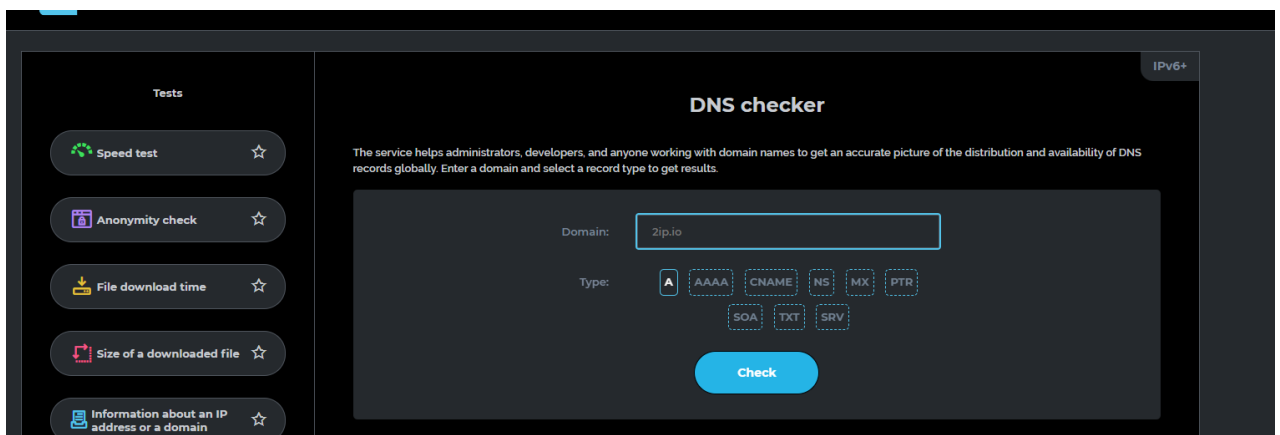
Скриншот: test2_speed_page.png

Тест 3: При попытке зайти на /invalidpage сервер выдал ошибку или перенаправил на главную. Скрипт зафиксировал изменение состояния. Статус: PASSED.

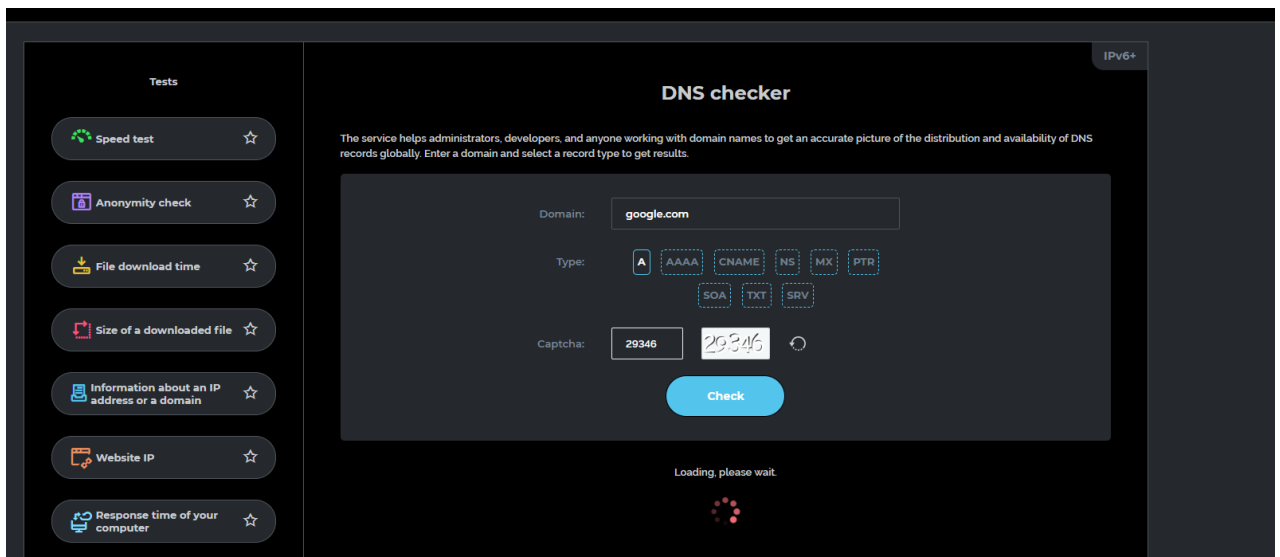


Скриншот: test3_404.png

Тест 4: Скрипт успешно нашел поле ввода на странице DNS-checker, ввел домен google.com и нажал кнопку проверки. После ожидания (около 15 секунд) были получены результаты сетевых записей. Статус: PASSED.



Скриншот: test4_dns_before.png



Скриншот: test4_dns_success.png

Консольный лог:

```
"C:\Users\migol\PycharmProjects\Tennis
Obus\venv\Lab_3\Scripts\python.exe" "C:/Program Files/JetBrains/PyCharm
Community Edition 2023.3.1/plugins/python-
ce/helpers/pycharm/_jb_pytest_runner.py" --path
C:\Users\migol\PycharmProjects\Lab_3\tests\test_2ip.py
Testing started at 10:58 ...
Launching pytest with arguments
C:\Users\migol\PycharmProjects\Lab_3\tests\test_2ip.py --no-header --
no-summary -q in C:\Users\migol\PycharmProjects\Lab_3\tests

===== test session starts
=====
collecting ... collected 4 items

test_2ip.py::test_main_page PASSED [
25%]Скриншот сохранён: screenshots\test1_main_page.png
Тест главной страницы пройден

test_2ip.py::test_speed_page PASSED [
50%]Скриншот сохранён: screenshots\test2_speed_page.png
Тест страницы скорости пройден

test_2ip.py::test_404_page PASSED [
75%]Скриншот сохранён: screenshots\test3_404.png
Тест 404 страницы пройден
```


test_2ip.py::test_dns_page_error PASSED
[100%]Открываем: https://2ip.io/dns-checker/
Скриншот сохранён: screenshots\test4_dns_before.png
Заголовок: DNS checker
URL: https://2ip.io/dns-checker/
Найдено input[type='submit']: 2
Submit 1: value='Login' | visible=False
Submit 2: value='Check' | visible=True
Ввели домен: google.com
Клик выполнен
Скриншот сохранён: screenshots\test4_dns_success.png
Тест DNS страницы пройден!

```
===== 4 passed in 58.05s =====
```

Process finished with exit code 0

Листинг программы:

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import WebDriverException,
TimeoutException, NoSuchElementException

import time
import os

# ===== НАСТРОЙКИ =====
BASE_URL = "https://2ip.io/"
SCREENSHOTS_DIR = "screenshots"

# Создаём папку для скриншотов
os.makedirs(SCREENSHOTS_DIR, exist_ok=True)

def get_driver(headless=False):
    """Создаём один драйвер с хорошими настройками"""
    chrome_options = Options()

    if headless:
        chrome_options.add_argument("--headless=new")

    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
```

```

        chrome_options.add_argument("--window-size=1920,1080")
        chrome_options.add_argument("--disable-blink-features=AutomationControlled")
        chrome_options.add_experimental_option("excludeSwitches",
["enable-automation"])
        chrome_options.add_experimental_option('useAutomationExtension',
False)

        driver = webdriver.Chrome(options=chrome_options)
        driver.execute_script("Object.defineProperty(navigator,
'webdriver', {get: () => undefined})")
        return driver

```

```

def take_screenshot(driver, name: str):
    """Сохраняем скриншот"""
    path = os.path.join(SCREENSHOTS_DIR, f"{name}.png")
    driver.save_screenshot(path)
    print(f"Скриншот сохранён: {path}")

```

```

# ===== ТЕСТЫ =====

```

```

def test_main_page():
    """Тест главной страницы"""
    driver = get_driver()
    try:
        driver.get(BASE_URL)

        # Явное ожидание
        WebDriverWait(driver, 10).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"),
"IP")

        )

        take_screenshot(driver, "test1_main_page")
        print("Тест главной страницы пройден")

    finally:
        driver.quit()

```

```

def test_speed_page():
    """Тест страницы скорости"""
    driver = get_driver()
    try:
        driver.get(BASE_URL)
        driver.get(f"{BASE_URL}speed/")

        WebDriverWait(driver, 8).until(
            lambda d: "speed" in d.current_url

```

```

    )

    take_screenshot(driver, "test2_speed_page")
    print("Тест страницы скорости пройден")

finally:
    driver.quit()

def test_404_page():
    """Тест несуществующей страницы"""
    driver = get_driver()
    try:
        driver.get(f"{BASE_URL}invalidpage")

        WebDriverWait(driver, 10).until(
            lambda d: "404" in d.page_source or d.current_url !=
f"{BASE_URL}invalidpage"
        )

        take_screenshot(driver, "test3_404")
        print("Тест 404 страницы пройден")

    finally:
        driver.quit()

def test_dns_page_error():
    """Тест страницы DNS checker"""
    driver = get_driver(headless=False) # Оставляем видимым для
отладки
    try:
        url = f"{BASE_URL}dns-checker/"
        print(f"Открываем: {url}")
        driver.get(url)

        time.sleep(5) # Даём время на загрузку (сайт тяжёлый)

        take_screenshot(driver, "test4_dns_before")

        print(f"Заголовок: {driver.title}")
        print(f"URL: {driver.current_url}")

        # === Ищем элементы отправки формы (input type=submit) ===
        submit_elements = driver.find_elements(By.XPATH,
"//input[@type='submit']")
        print(f"Найдено input[type='submit']: {len(submit_elements)}")

        for i, el in enumerate(submit_elements):
            value = el.get_attribute("value") or
el.get_attribute("name") or "No value"

```

```

        print(f" Submit {i + 1}: value='{value}' |
visible={el.is_displayed()}")

    # Берём первый видимый submit
    button = None
    for el in submit_elements:
        if el.is_displayed() and el.is_enabled():
            button = el
            break

    if not button:
        # Альтернативный поиск любой кнопки/ссылки
        button = driver.find_element(By.XPATH,
            "//*[contains(text(), 'Проверить') or contains(text(), 'Check')]")

    # Заполняем домен
    try:
        domain_input = driver.find_element(By.XPATH,
            "///input[contains(@placeholder, '2ip.io') or @type='text' and
            not(@type='hidden')]")
        domain_input.clear()
        domain_input.send_keys("google.com")
        print("Ввели домен: google.com")
    except:
        print("Не удалось найти поле для домена")

    # Кликаем
    button.click()
    print("Клик выполнен")

    # Ждём результат
    WebDriverWait(driver, 20).until(
        lambda d: any(word in d.page_source.lower() for word in
            ["error", "ошибк", "result", "результат",
            "ttl", "record", "a ", "aaaa", "ns ", "mx "])
    )

    take_screenshot(driver, "test4_dns_success")
    print("Тест DNS страницы пройден!")

except Exception as e:
    take_screenshot(driver, "test4_dns_error_FAILED")
    print(f"Ошибка: {type(e).__name__}: {e}")
    raise
finally:
    driver.quit()

# ===== ЗАПУСК =====

```

```
if __name__ == "__main__":
    print("Запуск тестов...\n")

    tests = [
        test_main_page,
        test_speed_page,
        test_404_page,
        test_dns_page_error
    ]

    for test in tests:
        try:
            test()
        except Exception as e:
            print(f"Ошибка в тесте {test.__name__}: {e}")
```